

SOFTWARE DEVELOPMENT • FULL-STACK • PERSONAL PROJECT

Ship a Portfolio Project That Gets You Hired

A complete, AI-assisted, start-to-finish build path — from idea validation to a deployed, promoted project — with the right tool at every stage and nothing left to guesswork.

END-TO-END PIPELINE — CLICK ANY STAGE TO JUMP

STAGE 1
Idea & Scope

STAGE 2
Design &
Architecture

STAGE 3
Environment
& Scaffolding

STAGE 4
Core
Development

STAGE 5
Testing &
Quality

STAGE 6
Deployment
& Docs

🕒 2–4 hrs estimated

📍 Stage 1 of 7

◆ OBJECTIVES

- Pick a project idea that signals the exact role you want (frontend, backend, full-stack)
- Define the problem it solves and who it's for — not just 'a to-do app'
- Set a scope you can finish in 1–3 weeks, not 3 months

◆ TASKS

- List 5 project ideas tied to jobs you're targeting
- Score each on: uniqueness, resume-relevance, buildability
- Write a one-paragraph problem statement and target user
- Define an MVP feature list and a 'v2 later' parking lot

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
Claude Ideation & scoping	Best at structured reasoning — can compare project ideas against your target job description and flag scope creep before you write code.	ChatGPT, Perplexity
Perplexity Market/competitor scan	Fast, cited web research to see what similar projects already exist so yours has a distinct angle.	Google + Claude summarizer
Notion AI Scope documentation	Keeps the brief, MVP list, and parking-lot ideas in one living doc you'll reference all project long.	Claude Projects, Obsidian

◆ PROMPT EXAMPLES

I want a portfolio project for a [frontend/backend/full-stack] developer role at [type of company]. Suggest 5 project ideas that are more distinctive than a to-do list or weather app, each solving a real, narrow problem. For each, list the core features and why it would stand out to a hiring manager.

Here is my project idea: [paste idea]. Write a one-paragraph problem statement, define the target user, and split features into 'MVP (ship in 2 weeks)' and 'v2 (later)'. Be ruthless about cutting scope.

✓ BEST PRACTICES

Pick something you'd actually use — it keeps you motivated past week one

Bias toward a project that lets you demonstrate ONE hard technical decision (e.g., real-time sync, auth, a non-trivial data model)

Write the MVP list before touching any code

✕ COMMON MISTAKES

Choosing a project that's a solved tutorial clone (Netflix clone #4,857)

Scoping something that needs 10 features to feel 'done'

Skipping the problem statement and jumping straight to UI

◆ EXPECTED OUTPUTS — CHECKLIST

☒ 1-page project brief

☒ MVP feature list

☒ v2 parking lot doc

⚡ EFFICIENCY TIP

Timebox ideation to one sitting — decision fatigue kills momentum more than a 'good enough' idea does.

📄 NOTES & BOOKMARKS

- Workflow visualization
- AI tool recommendations
- Prompt library
- Automation suggestions

3–6 hrs estimated

Stage 2 of 7

◆ OBJECTIVES

- Decide the tech stack with a reason for each choice
- Design the data model and core screens before coding
- Produce a visual direction so the UI doesn't look like a template

◆ TASKS

- Choose frontend framework, backend/API approach, database, hosting
- Sketch the data model (entities, relationships)
- Wireframe 3–5 core screens
- Define a visual identity: palette, type, one signature UI moment

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
v0 by Vercel UI scaffolding & layout ideas	Generates real React/Tailwind component code from a text prompt, so you get a working starting layout, not just a static mockup.	Claude Artifacts, Galileo AI
Claude (Artifacts) Data modeling & architecture reasoning	Can reason through entity relationships and stack tradeoffs conversationally, and render an interactive ER diagram or component tree directly in chat.	dbdiagram.io, Excalidraw
Figma (+ AI plugins) Polished wireframes	Industry-standard for screen design; AI plugins (Diagram, Magician) speed up layout variations.	Penpot (free/open-source)

◆ PROMPT EXAMPLES

Given this MVP feature list: [paste], recommend a tech stack (frontend, backend, database, hosting) suited to a solo developer shipping in 2 weeks. Justify each choice against alternatives and flag anything likely to cause pain later.

Design a data model for [project description]. List entities, fields, and relationships, and call out one modeling decision that's genuinely non-trivial so I can highlight it in interviews.

Propose a distinctive visual direction for [project] — not a generic SaaS-blue-and-white theme. Suggest a palette (with hex values), a type pairing, and one signature interaction or layout element that fits the subject.

✓ BEST PRACTICES

- Choose boring, well-documented tech for anything that isn't your 'hero' feature
- Design the data model before any UI — screens follow from data, not the reverse
- Pick ONE visual risk to take intentionally rather than several timid ones

✗ COMMON MISTAKES

- Chasing the newest framework instead of one you can debug quickly
- Designing pixel-perfect screens for features that might get cut
- Copy-pasting a generic dashboard template with no visual point of view

◆ EXPECTED OUTPUTS — CHECKLIST

- ☒ Stack decision doc
- ☒ ER diagram
- ☒ 3–5 wireframes
- ☒ Design tokens (colors/type/spacing)

 EFFICIENCY TIP

Generate 2–3 layout variants with v0/Claude in parallel, then pick — don't design from a blank canvas.

 NOTES & BOOKMARKS

Design:

- Responsive layout
- Dark theme
- Modern cards

 1–2 hrs estimated

 Stage 3 of 7

◆ OBJECTIVES

- Get a working local dev environment and repo in minutes, not hours
- Establish project structure, linting, and CI early
- Make the first commit a real, running 'hello world' of your stack

◆ TASKS

- Scaffold the project (framework CLI or AI-generated boilerplate)
- Set up Git repo, README skeleton, .env handling
- Configure linting/formatting and a basic CI check
- Deploy an empty shell to hosting to remove deploy-day surprises

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
Claude Code Full scaffolding + repo setup	Can generate the entire project skeleton, config files, and Git setup from a single instruction, and explain each generated file.	GitHub Copilot CLI, Cursor
Cursor AI-native IDE for the whole build	Codebase-aware autocomplete and chat that understands your actual files, not just isolated snippets — useful for the rest of the build too.	VS Code + Copilot, Windsurf
Vercel/Netlify CLI Instant deploy shell	One command gets a live URL before you've written a feature — removing deployment risk from the end of the project.	Railway, Render

◆ PROMPT EXAMPLES

```
Scaffold a [stack] project named [name] with this folder structure: [paste]. Include ESLint/Prettier config, a .env.example, and a GitHub Actions workflow that runs lint + build on push.
```

Write a README skeleton for this project with sections: overview, tech stack, local setup, environment variables, and screenshots placeholder.

✓ BEST PRACTICES

Deploy the empty scaffold on day one so 'it deploys' is never a surprise later

Commit early and often with meaningful messages — your history is part of the portfolio

Add lint/format hooks before the codebase grows

✗ COMMON MISTAKES

Writing weeks of code before the first deploy

Skipping .env.example, leaving future-you (or reviewers) guessing at config

No README until the very end

◆ EXPECTED OUTPUTS – CHECKLIST

☒ Running local dev environment

☒ Git repo with CI

☒ Live empty deploy URL

☒ README v0

⚡ EFFICIENCY TIP

Let an AI IDE generate boilerplate configs — hand-writing eslint/CI config is time better spent on your actual feature.

**NOTES & BOOKMARKS**

- Local HTML application
- GitHub repository
- README
- Ready for deployment

 20–50 hrs estimated

 Stage 4 of 7

◆ OBJECTIVES

- Build the MVP feature-by-feature against the plan from Stage 2
- Keep components small, typed, and testable as you go
- Use AI as a pair programmer without losing understanding of your own code

◆ TASKS

- Implement data layer / API endpoints first
- Build core UI components, then wire them to data
- Implement auth, if applicable, early — it touches everything else
- Handle loading, empty, and error states for every screen

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
Cursor / Windsurf In-editor AI pair programming	Reads your whole codebase for context-aware suggestions and multi-file edits, far more accurate than copy-pasting into a chat window.	GitHub Copilot, Claude Code
Claude Code Complex refactors & multi-file features	Strong at holding a large feature's context across many files and explaining tradeoffs as it implements — good for the trickiest 20% of the app.	Aider, Cursor Agent mode
Claude (chat) Debugging & 'why is this broken' sessions	Explaining a bug out loud to a reasoning model often surfaces the fix faster than a search-engine debugging loop.	Stack Overflow, ChatGPT

◆ PROMPT EXAMPLES

Here's my data model: [paste]. Generate the API endpoints for [feature] following REST conventions, with input validation and error responses.

Implement [component name] in [framework]. It should handle loading, empty, and error states, and match this design spec: [paste tokens/wireframe description].

Here's the error I'm getting: [paste stack trace + relevant code]. Walk me through likely causes before suggesting a fix, and explain why it's happening, not just the patch.

✓ BEST PRACTICES

Build vertical slices (one full feature end-to-end) rather than all UI then all backend

Read every AI-generated line before accepting it — you'll be asked to explain this code in interviews

Commit after each working slice, not at the end of the day

✗ COMMON MISTAKES

Accepting AI code you don't understand — it will show in a technical interview

Building every screen before wiring any real data

Ignoring error/empty states until 'later' (they rarely get done later)

◆ EXPECTED OUTPUTS — CHECKLIST

☒ Working MVP with core features

☒ API layer

☒ Component library

☒ Meaningful commit history

 EFFICIENCY TIP

Use AI for boilerplate and repetitive CRUD; spend your own attention on the one hard feature that's your interview talking point.

 NOTES & BOOKMARKS

Jot down anything specific to your project here...

 4–8 hrs estimated

 Stage 5 of 7

◆ OBJECTIVES

- Catch bugs before a hiring manager does
- Add enough automated tests to prove engineering discipline
- Check accessibility, performance, and responsive behavior

◆ TASKS

- Write unit tests for core logic and API endpoints
- Add a few integration/E2E tests for critical user flows
- Run a Lighthouse/accessibility audit and fix top issues
- Manually test on mobile viewport and slow network

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
Claude / Copilot Test generation	Can generate unit and integration tests from existing code quickly, including edge cases you might not think of.	Cursor test generation
Playwright (+ AI codegen) E2E testing	Recorder + AI-assisted codegen makes writing browser tests for critical flows (signup, checkout, core action) fast even if you're new to E2E.	Cypress
Lighthouse / axe DevTools Performance & accessibility audit	Free, built into Chrome, gives a concrete scored checklist — a strong 90+ score is an easy portfolio talking point.	WebPageTest

◆ PROMPT EXAMPLES

Here is [function/endpoint code]. Write unit tests covering the happy path, edge cases, and error handling using [testing framework].

Here's my critical user flow: [describe, e.g., sign up → create item → see it on dashboard]. Write a Playwright E2E test for this flow.

My Lighthouse report shows these issues: [paste]. Prioritize which to fix first for the best score-to-effort ratio and show the fix for the top 3.

✓ BEST PRACTICES

Test behavior, not implementation — tests should survive refactors

Prioritize E2E coverage on the flow a recruiter would actually click through

Fix accessibility basics (labels, contrast, focus states) — easy points, often skipped by other candidates

✗ COMMON MISTAKES

Zero tests, hoping no one asks about testing strategy in the interview

Testing trivial getters/setters while ignoring the core business logic

Never checking the site on an actual mobile device

◆ EXPECTED OUTPUTS — CHECKLIST

✓ Unit test suite

✓ 1–3 E2E tests on critical flows

✓ Lighthouse report ≥ 90

✓ Mobile QA pass

 EFFICIENCY TIP

Generate tests immediately after writing a function, while the intended behavior is still fresh — don't batch testing for the end.

 NOTES & BOOKMARKS

Jot down anything specific to your project here...

 2–4 hrs estimated

 Stage 6 of 7

◆ OBJECTIVES

- Ship to production with a real domain and monitoring
- Make the repo and live demo genuinely reviewable by a stranger
- Write documentation that shows how you think, not just what you built

◆ TASKS

- Deploy to production hosting with environment variables set
- Connect a custom or clean subdomain, enable HTTPS
- Write a complete README: overview, stack, architecture decisions, setup, screenshots
- Add basic error monitoring/analytics

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
Vercel / Netlify Frontend/full-stack hosting	Zero-config deploys from Git with preview URLs per branch — makes iteration and demoing effortless.	Railway, Render, Fly.io
Claude README & architecture write-up	Turns your rough notes and code into a clear, recruiter-readable README, including an 'architecture decisions' section that shows engineering judgment.	ChatGPT, Notion AI
Sentry (free tier) Error monitoring	Shows you actually think about production reliability — a small addition with outsized interview value.	LogRocket, PostHog

◆ PROMPT EXAMPLES

Here are my rough notes and file structure: [paste]. Write a polished README with sections: Overview, Live Demo, Tech Stack, Architecture Decisions (with reasoning), Local Setup, Screenshots, and Roadmap.

Write a 3-sentence 'architecture decisions' summary explaining why I chose [stack/pattern] over [alternative], suitable for a README and for repeating verbally in an interview.

✓ BEST PRACTICES

Put the live demo link at the very top of the README, above the fold

Include 2–3 real screenshots or a short GIF, not just text

Write the 'why' behind key decisions — this is what differentiates your project in review

✗ COMMON MISTAKES

A live link that 404s because env vars weren't set on the host

A README that's just 'npm install && npm run dev' with nothing else

No screenshots — reviewers often won't run the project locally

◆ EXPECTED OUTPUTS – CHECKLIST

☒ Live production URL

☒ Polished README

☒ Architecture decisions doc

☒ Basic monitoring dashboard

⚡ EFFICIENCY TIP

Draft the README from your own commit messages and notes with AI — you'll have already written 80% of the content without realizing it.



NOTES & BOOKMARKS

Jot down anything specific to your project here...



 Ongoing, 2–3 hrs/wk estimated

 Stage 7 of 7

◆ OBJECTIVES

- Get the project seen by the people who can hire you
- Turn the build into interview-ready talking points
- Keep iterating based on real feedback

◆ TASKS

- Write a short build-in-public post (LinkedIn/X) with the live link
- Add the project to your resume and portfolio site with a 2-line summary
- Prepare 3 STAR-format stories about decisions made during the build
- Ship one visible v2 improvement based on feedback

◆ BEST AI TOOLS FOR THIS STAGE

TOOL	WHY IT'S RECOMMENDED	ALTERNATIVES
Claude Interview story prep & post drafting	Helps turn raw build notes into concise STAR-format interview answers and platform-appropriate promo posts.	ChatGPT
Loom Demo video walkthrough	A 60–90 second narrated walkthrough dramatically increases how many reviewers actually engage with your project.	OBS Studio + editing
Readymag / your portfolio site Project case study page	A dedicated case-study page (problem → decisions → result) reads as more senior than a bare GitHub link.	Notion public page, personal site

◆ PROMPT EXAMPLES

Turn these build notes into a LinkedIn post announcing my project, written in a genuine, not-salesy voice, ending with the live demo link.

Here are 3 tricky decisions I made while building [project]: [list them]. Turn each into a STAR-format interview story (Situation, Task, Action, Result).

Based on this feedback I received: [paste], suggest 2–3 v2 improvements ranked by impact-to-effort for a solo developer.

✓ BEST PRACTICES

Ship a visible, dated v2 update — it signals the project (and you) are alive and improving

Prepare your 'why' for every non-obvious decision before interviews, not during them

Ask 2–3 people outside your bubble to try the live demo and report friction

✗ COMMON MISTAKES

Posting once and never following up on comments or feedback

Letting the project go stale with an unaddressed open issue for months

Memorizing a script instead of understanding the decisions well enough to improvise

◆ EXPECTED OUTPUTS – CHECKLIST

☒ Promo post

☒ Case study page

☒ 3 STAR interview stories

☒ Shipped v2 improvement

 EFFICIENCY TIP

Batch interview-story prep right after deployment, while the reasoning behind decisions is still top of mind.

 NOTES & BOOKMARKS

Jot down anything specific to your project here...

◆ WORKFLOW SUMMARY

This workflow takes a full-stack portfolio project from a validated idea to a live, promoted deployment across 7 stages — Idea & Scope, Design & Architecture, Environment & Scaffolding, Core Development, Testing & Quality, Deployment & Docs, and Promotion & Iteration. Total realistic effort: roughly **35–75 hours** across 1–3 weeks for a solo developer using AI tools throughout, from scoping through interview-ready promotion.

◆ RECOMMENDED AI STACK

REASONING & PLANNING

Claude

IN-EDITOR CODING

Cursor

MULTI-FILE AGENTIC CODING

Claude Code

UI SCAFFOLDING

v0 by Vercel

RESEARCH

Perplexity

HOSTING

Vercel / Netlify

MONITORING

Sentry (free)

DEMO VIDEO

Loom

◆ LEARNING RESOURCES

- Official docs for your chosen framework — always the highest-signal source
- "Refactoring UI" (Adam Wathan & Steve Schoger) for interface design instincts
- web.dev — performance, accessibility, and Lighthouse deep-dives
- Your framework's official Discord/forum for scaffold-specific issues

◆ COMMUNITIES

- r/webdev, r/cscareerquestions for feedback and job-market signal
- Indie Hackers for build-in-public accountability
- Local or virtual dev meetups for live project demos

- Discord servers for your specific framework/stack

◆ SEARCH KEYWORDS

portfolio project ideas developer

full-stack project architecture

README best practices github

AI pair programming workflow

deploy full-stack app free tier

build in public developer

STAR method interview coding project

lighthouse accessibility audit fix

◆ ADDITIONAL AI PROMPTS

Act as a senior engineer reviewing my portfolio project's codebase for a hiring evaluation. Here's the repo structure and key files: [paste]. Give direct, specific feedback on what would raise or lower your confidence in hiring me.

Simulate a technical interviewer asking me to walk through this project's architecture. Ask 5 progressively harder questions, one at a time, and critique my answers.

◆ FUTURE AUTOMATION OPPORTUNITIES

- Auto-generate changelogs from commit messages using an AI CI step
- Scheduled AI-written weekly "build log" post drafts during active development
- Automated Lighthouse/accessibility regression checks on every deploy
- AI-generated release notes and social posts triggered on each tagged version

◆ DECISION TREE – "WHICH STACK SHOULD I PICK?"

Do you want to emphasize frontend polish or backend/data complexity?

